LAPORAN PRAKTIKUM
ALGORITMA & STRUKTUR DATA
MODUL 2

STACK DAN QUEUE

Oleh:

MUHAMMMAD RAFLY ASH-SHADIQ NIM. 2510817310007

PROGRAM STUDI TEKNOLOGI INFORMASI
FAKULTAS TEKNIK
UNIVERSITAS LAMBUNG MANGKURAT
APRIL 2026

LEMBAR PENGESAHAN
LAPORAN PRAKTIKUM ALGORITMA & STRUKTUR DATA
MODUL 2

Laporan Praktikum Algoritma & Struktur Data Modul 2: Stack dan Queue ini disusun sebagai syarat lulus mata kuliah Praktikum Algoritma & Struktur Data. Laporan Prakitkum ini dikerjakan oleh:

Nama Praktikan : Muhammad Rafly Ash-Shadiq
NIM : 2510817310007

Menyetujui,	Mengetahui,
Asisten Praktikum	Dosen Penanggung Jawab Praktikum
Fadhil Syahdama Mahatma Putra	Andreyan Rizky Baskara, S.Kom., M.Kom
NIM. 2410817210026	NIP. 199307032019031011

DAFTAR ISI

LEMBAR PENGESAHAN i
DAFTAR ISI ii
DAFTAR GAMBAR iii
DAFTAR TABEL iv
SOAL 1 1
A. Pembahasan 1
SOAL 2 3
A. Source Code 4
B. Output Program 7
C. Pembahasan 7
SOAL 3 11
A. Source Code 11
B. Output Program 14
C. Pembahasan 15
LINK GITHUB 18

DAFTAR GAMBAR

Gambar 1 Screenshot Output Program Soal 2 7
Gambar 2 Screenshot Output Program Soal 3 14

DAFTAR TABEL

Tabel 1 Source Code Soal Nomor 2 4
Tabel 2 Source Code Soal Nomor 3 11

SOAL 1

Apa perbedaan Stack dengan Queue?

A. Pembahasan

Program Stack (tumpukan) dan Queue (antrian) merupakan dua jenis struktur data linear yang digunakan untuk menyimpan data secara berurutan, namun keduanya memiliki prinsip kerja yang berbeda. Stack bekerja dengan konsep LIFO (Last In First Out), yaitu data yang terakhir dimasukkan akan menjadi data pertama yang dikeluarkan. Konsep ini dapat dianalogikan seperti tumpukan buku, di mana buku yang terakhir diletakkan di bagian atas akan menjadi yang pertama diambil. Dalam Stack, proses penambahan data disebut dengan push dan hanya dilakukan pada satu sisi, yaitu bagian atas (top), sedangkan proses pengambilan data disebut dengan pop yang juga dilakukan dari sisi yang sama.

Berbeda dengan Stack, Queue (antrian) menggunakan konsep FIFO (First In First Out), yaitu data yang pertama dimasukkan akan menjadi data pertama yang dikeluarkan. Konsep ini serupa dengan antrian dalam kehidupan sehari-hari, seperti antrian di loket, di mana orang yang datang lebih dahulu akan dilayani lebih dahulu. Pada Queue, proses penambahan data disebut enqueue dan dilakukan dari bagian belakang (rear), sedangkan proses pengambilan data disebut dequeue dan dilakukan dari bagian depan (front). Hal ini membuat Queue memiliki dua sisi akses, yaitu depan dan belakang.

Secara algoritma, Stack memiliki alur kerja yang sederhana, dimulai dari kondisi kosong, kemudian setiap data yang masuk akan ditumpuk di atas data sebelumnya. Ketika dilakukan pengambilan data, sistem akan mengambil data paling atas terlebih dahulu. Proses ini akan terus berlangsung hingga Stack kosong. Jika terjadi pengambilan data saat Stack kosong, maka akan terjadi kesalahan karena tidak ada data yang dapat diambil. Sebaliknya, Queue memiliki alur kerja yang mengutamakan urutan kedatangan. Data yang masuk akan ditempatkan di bagian belakang, kemudian saat pengambilan data, sistem akan mengambil dari bagian depan. Setelah satu data keluar, data berikutnya akan bergeser ke depan. Jika pengambilan dilakukan saat Queue kosong, maka sistem juga tidak dapat mengambil data.

Dengan demikian, perbedaan utama antara Stack dan Queue terletak pada cara pengolahan datanya. Stack lebih menekankan pada data terakhir yang masuk, sedangkan Queue lebih menekankan pada urutan kedatangan data. Stack cocok digunakan pada kondisi yang membutuhkan pembalikan urutan data, sedangkan Queue lebih sesuai digunakan pada sistem antrian atau proses yang berjalan secara berurutan.

SOAL 2

Cobalah contoh program berikut, running dan analisis hasilnya!

```
int penuh()
{
    if(Tumpuk.atas == max-1)
        return 1;
    else
        return 0;
}

void input (int data)
{
    if (kosong()==1)
    {
        Tumpuk.atas++;
        Tumpuk.data[Tumpuk.atas] = data;
        cout << "Data " << Tumpuk.data[Tumpuk.atas]
             << " Masuk Ke Stack ";
    }
    else if(penuh()==0)
    {
        Tumpuk.atas++;
        Tumpuk.data[Tumpuk.atas] = data;
        cout << "Data "<< Tumpuk.data[Tumpuk.atas]
             << " Masuk Ke Stack ";
    }
    else
        cout << "Tumpukan Penuh";
}
```

```
void hapus()
{
    if(kosong()== 0)
    {
        cout << "Data Teratas Sudah Terambil";
        Tumpuk.atas--;
    }
    else
    cout <<" Data Kosong";
}
void tampil()
{
    if (kosong()== 0)
    {
        for(int i = Tumpuk.atas; i>=0; i--)
        {
            cout <<"\nTumpukan Ke " << i << " = "
            <<Tumpuk.data[i];
        }
    }
    else
    cout << "Tumpukan Kosong";
}
void bersih ()
{
    Tumpuk.atas = -1;
    cout << "Tumpukan Kosong !";
}
```

A. Source Code

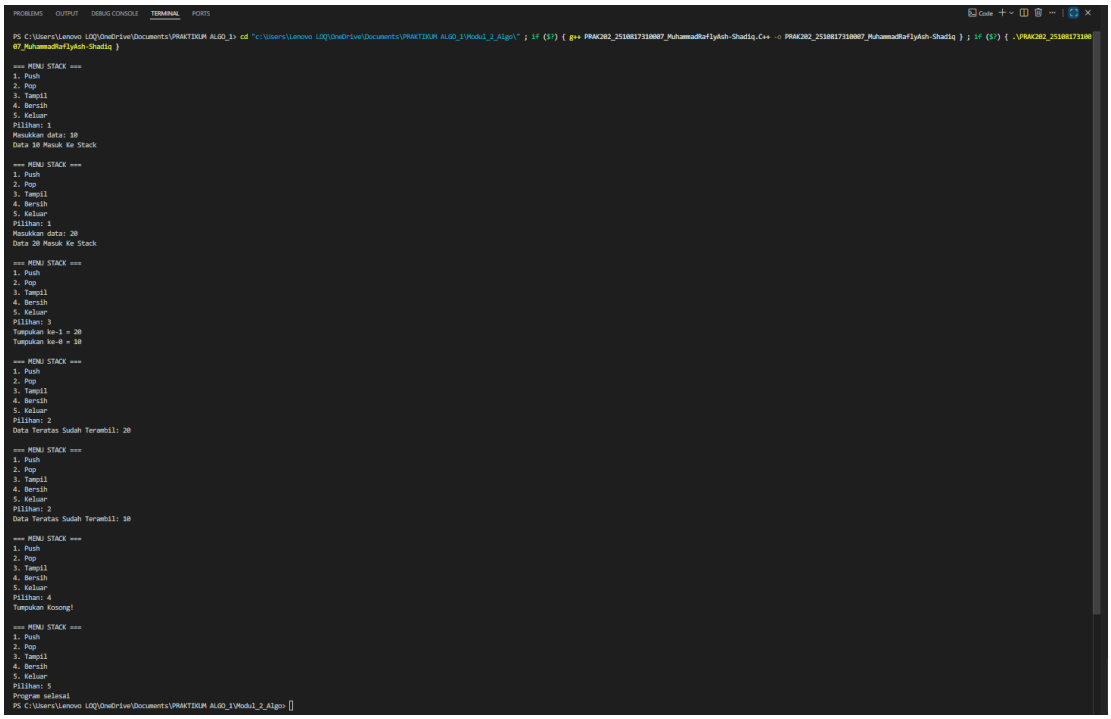
Tabel 1 Source Code Soal Nomor 2

```
#include <iostream>
using namespace std;
#define max 5

struct stack {
    int data[max];
    int atas;
};

stack Tumpuk;
int kosong() {
    if (Tumpuk.atas == -1)
        return 1;
    else
        return 0;
}

int penuh() {
    if (Tumpuk.atas == max - 1)
        return 1;
    else
        return 0;
}

void input(int data) {
    if (kosong() == 1) {
        Tumpuk.atas++;
        Tumpuk.data[Tumpuk.atas] = data;
        cout << "Data " << Tumpuk.data[Tumpuk.atas]
             << " Masuk Ke Stack\n";
    }
    else if (penuh() == 0) {
        Tumpuk.atas++;
        Tumpuk.data[Tumpuk.atas] = data;
        cout << "Data " << Tumpuk.data[Tumpuk.atas]
             << " Masuk Ke Stack\n";
    }
```

```
    else {
        cout << "Tumpukan Penuh\n";
    }
}

void hapus() {
    if (kosong() == 0) {
        cout << "Data Teratas Sudah Terambil: "
             << Tumpuk.data[Tumpuk.atas] << endl;
        Tumpuk.atas--;
    }
    else {
        cout << "Data Kosong\n";
    }
}

void tampil() {
    if (kosong() == 0) {
        for (int i = Tumpuk.atas; i >= 0; i--) {
            cout << "Tumpukan ke-" << i << " = "
                 << Tumpuk.data[i] << endl;
        }
    }
    else {
        cout << "Tumpukan Kosong\n";
    }
}

void bersih() {
    Tumpuk.atas = -1;
    cout << "Tumpukan Kosong!\n";
}

int main() {
    int pilihan, data;

    Tumpuk.atas = -1;

    do {
        cout << "\n=== MENU STACK ===\n";
        cout << "1. Push\n";
```

```
79.         cout << "2. Pop\n";
80.         cout << "3. Tampil\n";
81.         cout << "4. Bersih\n";
82.         cout << "5. Keluar\n";
83.         cout << "Pilihan: ";
84.         cin >> pilihan;
85.
86.         switch (pilihan) {
87.             case 1:
88.                 cout << "Masukkan data: ";
89.                 cin >> data;
90.                 input(data);
91.                 break;
92.
93.             case 2:
94.                 hapus();
95.                 break;
96.
97.             case 3:
98.                 tampil();
99.                 break;
100.
101.            case 4:
102.                bersih();
103.                break;
104.
105.            case 5:
106.                cout << "Program selesai\n";
107.                break;
108.
109.            default:
110.                cout << "Pilihan tidak valid\n";
111.        }
112.
113.    } while (pilihan != 5);
114.
115.    return 0;
116. }
```

B. Output Program



```
PROBLEMS    OUTPUT    DEBUG CONSOLE    TERMINAL    PORTS

PS C:\Users\Lenovo LOQ\OneDrive\Documents\PRAKTIKUM ALGO_1> cd "c:\Users\Lenovo LOQ\OneDrive\Documents\PRAKTIKUM ALGO_1\Modul_2_Algo\" ; if ($?) { g++ PRAK202_2510817310007_MuhammadRaflyAsh-Shadiq.C++ -o PRAK202_2510817310007_MuhammadRaflyAsh-Shadiq } ; if ($?) { .\PRAK202_2510817310
07_MuhammadRaflyAsh-Shadiq }

=== MENU STACK ===
1. Push
2. Pop
3. Tampil
4. Bersih
5. Keluar
Pilihan: 1
Masukkan data: 10
Data 10 Masuk Ke Stack

=== MENU STACK ===
1. Push
2. Pop
3. Tampil
4. Bersih
5. Keluar
Pilihan: 1
Masukkan data: 20
Data 20 Masuk Ke Stack

=== MENU STACK ===
1. Push
2. Pop
3. Tampil
4. Bersih
5. Keluar
Pilihan: 3
Tumpukan ke-1 = 20
Tumpukan ke-0 = 10

=== MENU STACK ===
1. Push
2. Pop
3. Tampil
4. Bersih
5. Keluar
Pilihan: 2
Data Teratas Sudah Terambil: 20

=== MENU STACK ===
1. Push
2. Pop
3. Tampil
4. Bersih
5. Keluar
Pilihan: 2
Data Teratas Sudah Terambil: 10

=== MENU STACK ===
1. Push
2. Pop
3. Tampil
4. Bersih
5. Keluar
Pilihan: 4
Tumpukan Kosong!

=== MENU STACK ===
1. Push
2. Pop
3. Tampil
4. Bersih
5. Keluar
Pilihan: 5
Program selesai
PS C:\Users\Lenovo LOQ\OneDrive\Documents\PRAKTIKUM ALGO_1\Modul_2_Algo>
```

Gambar 1 Screenshot Output Program Soal 2

C. Pembahasan

Program yang dibuat merupakan implementasi struktur data Stack (tumpukan) menggunakan bahasa pemrograman C++. Stack bekerja dengan prinsip LIFO (Last In First Out), artinya data yang terakhir dimasukkan akan menjadi data pertama yang dikeluarkan. Pada program ini, data disimpan dalam sebuah array dengan kapasitas tertentu, dan posisi data teratas dikontrol menggunakan variabel atas.

Alur program dimulai dari inisialisasi stack dalam keadaan kosong, yaitu dengan nilai atas = -1. Selanjutnya, program menampilkan menu interaktif yang memungkinkan pengguna memilih operasi yang tersedia, yaitu input data (push), hapus data (pop), menampilkan isi stack, dan mengosongkan stack. Program akan terus berjalan selama pengguna tidak memilih keluar.

Pada proses input (push), pengguna memasukkan sebuah nilai yang kemudian akan dimasukkan ke dalam stack jika kondisi stack belum penuh. Jika stack penuh, maka program akan menampilkan pesan bahwa tumpukan sudah tidak dapat menampung data lagi. Sebaliknya, pada proses hapus (pop), program akan mengambil data paling atas dari stack dan menguranginya satu tingkat. Jika stack dalam kondisi kosong, maka program akan memberikan peringatan bahwa tidak ada data yang bisa diambil.

Untuk proses tampil, program akan menampilkan seluruh isi stack mulai dari data paling atas hingga paling bawah, sehingga pengguna dapat melihat urutan data sesuai konsep LIFO. Sedangkan pada proses bersih, seluruh data dalam stack akan dihapus dengan cara mengatur kembali nilai atas menjadi -1.

Berdasarkan hasil eksekusi pada gambar, terlihat bahwa program berjalan sesuai algoritma. Saat data dimasukkan (push), data akan bertambah dari bawah ke atas. Ketika dilakukan operasi pop, data yang terakhir dimasukkan akan dihapus terlebih dahulu. Hal ini menunjukkan bahwa implementasi stack pada program sudah sesuai dengan konsep dasarnya. Selain itu, program juga mampu menangani kondisi batas seperti stack penuh dan stack kosong, sehingga dapat dikatakan program berjalan dengan logika yang benar dan terstruktur.

Pada baris 1, terdapat #include <iostream> yang digunakan untuk mengaktifkan fungsi input dan output seperti cin dan cout. Pada baris 2, terdapat using namespace std; yang berfungsi agar penulisan tidak perlu menggunakan std:: sebelum cin dan cout. Pada baris 3, terdapat #define max 5 yang digunakan untuk menentukan kapasitas maksimum stack sebanyak 5 data. Pada baris 5–8, dideklarasikan struct stack yang berisi array data untuk menyimpan elemen stack dan variabel atas sebagai penunjuk posisi elemen paling atas. Pada baris 10, dibuat variabel global Tumpuk dari struct stack yang akan digunakan untuk menyimpan data selama program berjalan. Pada baris 11–16, terdapat fungsi kosong() yang berfungsi untuk mengecek apakah stack dalam keadaan kosong. Jika atas == -1, maka fungsi mengembalikan nilai 1 (benar), selain itu 0. Pada baris 18–23, terdapat fungsi penuh() yang digunakan untuk mengecek apakah

stack sudah penuh. Jika atas == max - 1, maka stack dianggap penuh. Pada baris 25, dideklarasikan fungsi input(int data) yang berfungsi untuk menambahkan data ke dalam stack (push). Pada baris 26–31, dilakukan pengecekan apakah stack kosong. Jika kosong, maka atas ditambah satu dan data dimasukkan ke dalam array, lalu ditampilkan ke layar. Pada baris 32–37, dilakukan pengecekan jika stack tidak penuh. Jika masih ada ruang, maka proses penambahan data tetap dilakukan seperti sebelumnya. Pada baris 38–40, jika stack sudah penuh, maka program menampilkan pesan "Tumpukan Penuh". Pada baris 43, dideklarasikan fungsi hapus() yang berfungsi untuk mengambil data dari stack (pop). Pada baris 44–48, jika stack tidak kosong, maka data paling atas ditampilkan lalu atas dikurangi satu untuk menghapus data tersebut. Pada baris 49–51, jika stack kosong, maka program menampilkan pesan "Data Kosong". Pada baris 54, dideklarasikan fungsi tampil() yang berfungsi untuk menampilkan seluruh isi stack. Pada baris 55–60, jika stack tidak kosong, maka dilakukan perulangan dari indeks atas sampai 0 untuk menampilkan data dari atas ke bawah. Pada baris 61–63, jika stack kosong, maka program menampilkan pesan "Tumpukan Kosong". Pada baris 66, dideklarasikan fungsi bersih() yang digunakan untuk mengosongkan stack. Pada baris 67–68, nilai atas diatur kembali menjadi -1 sehingga stack dianggap kosong, lalu ditampilkan pesan konfirmasi. Pada baris 71, program utama main() mulai dijalankan. Pada baris 72, dideklarasikan variabel pilihan untuk menu dan data untuk input pengguna. Pada baris 74, dilakukan inisialisasi awal dengan mengatur Tumpuk.atas = -1 agar stack dalam kondisi kosong saat program dimulai. Pada baris 76–113, digunakan perulangan do-while untuk menjalankan menu secara terus-menerus sampai pengguna memilih keluar. Pada baris 77–83, ditampilkan menu pilihan operasi stack seperti push, pop, tampil, bersih, dan keluar. Pada baris 84, pengguna memasukkan pilihan menu menggunakan cin. Pada baris 86–111, digunakan struktur switch-case untuk menjalankan perintah sesuai pilihan pengguna. Pada baris 87–91, jika memilih push, program meminta input data lalu memanggil fungsi input(). Pada baris 93–95, jika memilih pop, program memanggil fungsi hapus(). Pada baris 97–99, jika memilih tampil, program memanggil fungsi tampil(). Pada baris 101–103, jika memilih bersih, program memanggil fungsi bersih(). Pada baris 105–107, jika memilih keluar, program

menampilkan pesan bahwa program selesai. Pada baris 109–110, jika pilihan tidak sesuai, maka program menampilkan pesan kesalahan. Pada baris 113, perulangan akan terus berjalan selama pilihan tidak sama dengan 5 (keluar). Pada baris 115–116, program diakhiri dengan return 0 yang menandakan program selesai dijalankan dengan baik.

SOAL 3

Cobalah contoh program berikut, running dan analisis hasilnya!

```
#include<iostream>
#include<conio.h>
#include<stdlib.h>
#define n 10
using namespace std;

void INSERT();
void DELETE();
void CETAKLAYAR();
void Inisialisasi();
void RESET();
int PIL,F,R;
char PILIHAN [1],HURUF;
char Q[n];
int main ( )
{
     Inisialisasi();
     do
     {
           cout<<"QUEUE"<<endl;
           cout<<"==============="<<endl;
           cout<<"1. INSERT"<<endl;
           cout<<"2. DELETE"<<endl;
           cout<<"3. CETAK QUEUE"<<endl;
           cout<<"4. QUIT"<<endl;
           cout<<"PILIHAN : "; cin>>PILIHAN;
           PIL=atoi(PILIHAN);


        switch (PIL)
        {
        case 1:
             INSERT ();
             break;
        case 2:
             DELETE();
             break;
        case 3:
             CETAKLAYAR ();
             break;
        default:
             cout<<"TERIMA KASIH"<<endl;
             break;
        }
        cout<<"press any key to continue"<<endl;
        getch();
        system("cls");
     }
     while (PIL<4);
}
```

A. Source Code

Tabel 2 Source Code Soal Nomor 3

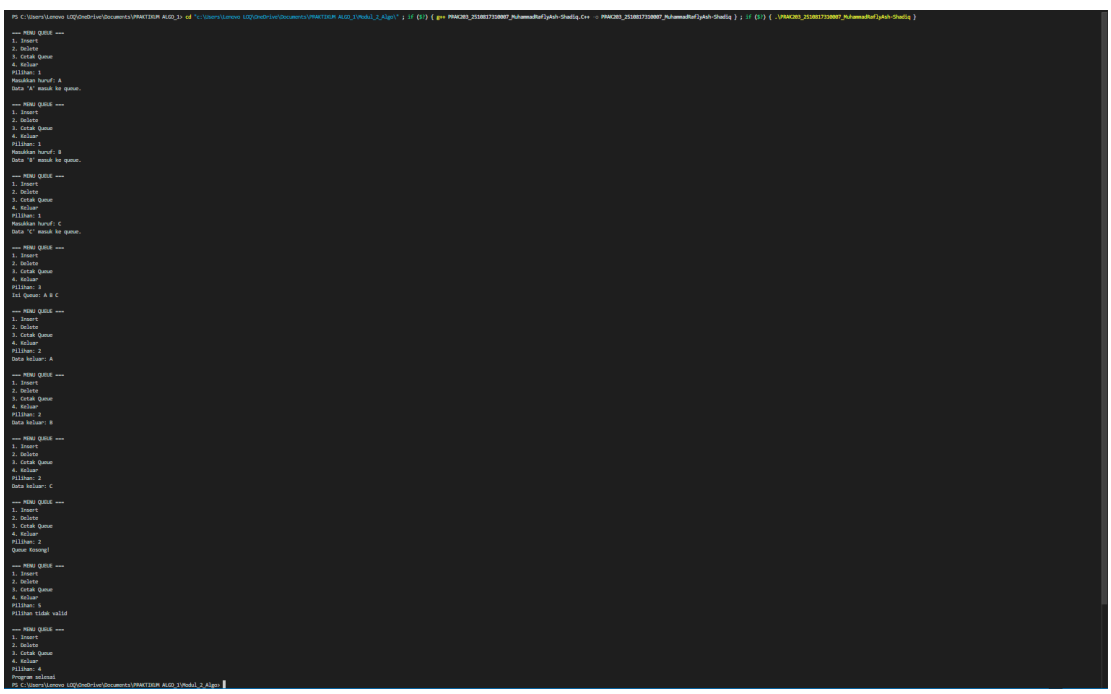
1.	`#include <iostream>`

```
using namespace std;
#define n 10

int F, R;
char Q[n];

void Inisialisasi() {
    F = -1;
    R = -1;
}

bool kosong() {
    return (F == -1 || F > R);
}

bool penuh() {
    return (R == n - 1);
}

void INSERT() {
    if (penuh()) {
        cout << "Queue Penuh!\n";
    } else {
        char huruf;
        cout << "Masukkan huruf: ";
        cin >> huruf;

        if (F == -1)
            F = 0;

        R++;
        Q[R] = huruf;

         cout << "Data '" << huruf << "' masuk ke queue.\n";
    }
}

void DELETE() {
    if (kosong()) {
        cout << "Queue Kosong!\n";
```

```
42.     } else {
43.         cout << "Data keluar: " << Q[F] << endl;
44.         F++;
45.
46.         if (F > R)
47.             Inisialisasi();
48.     }
49. }
50.
51. void CETAKLAYAR() {
52.     if (kosong()) {
53.         cout << "Queue Kosong!\n";
54.     } else {
55.         cout << "Isi Queue: ";
56.         for (int i = F; i <= R; i++) {
57.             cout << Q[i] << " ";
58.         }
59.         cout << endl;
60.     }
61. }
62.
63. int main() {
64.     int pilihan;
65.
66.     Inisialisasi();
67.
68.     do {
69.         cout << "\n=== MENU QUEUE ===\n";
70.         cout << "1. Insert\n";
71.         cout << "2. Delete\n";
72.         cout << "3. Cetak Queue\n";
73.         cout << "4. Keluar\n";
74.         cout << "Pilihan: ";
75.         cin >> pilihan;
76.
77.         switch (pilihan) {
78.             case 1:
79.                 INSERT();
80.                 break;
81.             case 2:
82.                 DELETE();
```

```
                    break;
                case 3:
                    CETAKLAYAR();
                    break;
                case 4:
                    cout << "Program selesai\n";
                    break;
                default:
                    cout << "Pilihan tidak valid\n";
            }

        } while (pilihan != 4);

        return 0;
}
```

B. Output Program



Gambar 2 Screenshot Output Program Soal 3

C. Pembahasan

Program yang dibuat merupakan implementasi struktur data Queue (antrian) menggunakan bahasa pemrograman C++. Queue bekerja dengan prinsip FIFO (First In First Out), artinya data yang pertama dimasukkan akan menjadi data pertama yang dikeluarkan. Pada program ini, data disimpan dalam sebuah array dengan kapasitas tertentu, dan posisi data dikontrol menggunakan dua variabel yaitu F (Front) dan R (Rear).

Alur program dimulai dari inisialisasi queue dalam keadaan kosong, yaitu dengan nilai F = -1 dan R = -1. Selanjutnya, program menampilkan menu interaktif yang memungkinkan pengguna memilih operasi yang tersedia, yaitu menambah data (insert), menghapus data (delete), serta menampilkan isi queue. Program akan terus berjalan selama pengguna tidak memilih keluar.

Pada proses input (insert), pengguna memasukkan sebuah huruf yang kemudian akan dimasukkan ke dalam queue jika kondisi queue belum penuh. Jika queue penuh, maka program akan menampilkan pesan bahwa antrian sudah tidak dapat menampung data lagi. Sebaliknya, pada proses hapus (delete), program akan mengambil data paling depan dari queue dan menggeser posisi front ke elemen berikutnya. Jika queue dalam kondisi kosong, maka program akan memberikan peringatan bahwa tidak ada data yang bisa dihapus.

Untuk proses tampil, program akan menampilkan seluruh isi queue mulai dari elemen paling depan hingga paling belakang, sehingga pengguna dapat melihat urutan data sesuai konsep FIFO. Selain itu, program juga melakukan reset otomatis jika seluruh data telah dihapus, sehingga queue kembali ke kondisi awal.

Berdasarkan hasil eksekusi pada gambar, terlihat bahwa program berjalan sesuai algoritma. Saat data dimasukkan (insert), data akan bertambah dari belakang. Ketika dilakukan operasi delete, data yang pertama dimasukkan akan dihapus terlebih dahulu. Hal ini menunjukkan bahwa implementasi queue pada program sudah sesuai dengan konsep dasarnya. Selain itu, program juga mampu menangani kondisi batas

seperti queue penuh dan queue kosong, sehingga dapat dikatakan program berjalan dengan logika yang benar dan terstruktur.

Pada baris 1, terdapat #include <iostream> yang digunakan untuk mengaktifkan fungsi input dan output seperti cin dan cout. Pada baris 2, terdapat using namespace std; yang berfungsi agar penulisan tidak perlu menggunakan std:: sebelum cin dan cout. Pada baris 3, terdapat #define n 10 yang digunakan untuk menentukan kapasitas maksimum queue sebanyak 10 data. Pada baris 5–6, dideklarasikan variabel global yaitu F dan R sebagai penunjuk posisi front dan rear, serta array Q[n] yang digunakan untuk menyimpan data queue. Pada baris 8–11, terdapat fungsi Inisialisasi() yang berfungsi untuk mengatur queue dalam kondisi kosong dengan nilai F = -1 dan R = -1. Pada baris 13–15, terdapat fungsi kosong() yang digunakan untuk mengecek apakah queue dalam keadaan kosong. Jika F == -1 atau F > R, maka queue dianggap kosong. Pada baris 17–19, terdapat fungsi penuh() yang digunakan untuk mengecek apakah queue sudah penuh. Jika R == n - 1, maka queue dianggap penuh. Pada baris 21, dideklarasikan fungsi INSERT() yang berfungsi untuk menambahkan data ke dalam queue. Pada baris 22–24, dilakukan pengecekan apakah queue penuh. Jika penuh, maka program menampilkan pesan "Queue Penuh". Pada baris 25–27, pengguna diminta memasukkan data berupa karakter. Pada baris 29–30, dilakukan pengecekan jika queue sebelumnya kosong, maka F diatur menjadi 0. Pada baris 32–33, nilai R ditambah satu dan data dimasukkan ke dalam array queue. Pada baris 35, program menampilkan pesan bahwa data berhasil dimasukkan. Pada baris 39, dideklarasikan fungsi DELETE() yang berfungsi untuk menghapus data dari queue. Pada baris 40–42, dilakukan pengecekan apakah queue kosong. Jika kosong, maka ditampilkan pesan "Queue Kosong". Pada baris 43–44, data pada posisi front ditampilkan lalu F ditambah satu untuk menghapus data tersebut. Pada baris 46–47, jika F > R, maka queue diinisialisasi kembali agar kembali ke kondisi kosong. Pada baris 51, dideklarasikan fungsi CETAKLAYAR() yang berfungsi untuk menampilkan isi queue. Pada baris 52–54, dilakukan pengecekan apakah queue kosong. Jika kosong, ditampilkan pesan "Queue Kosong". Pada baris 55–59, jika queue tidak kosong, maka dilakukan

perulangan dari indeks F sampai R untuk menampilkan seluruh isi queue. Pada baris 63, program utama main() mulai dijalankan. Pada baris 64, dideklarasikan variabel pilihan untuk menyimpan input menu dari pengguna. Pada baris 66, dilakukan inisialisasi queue dengan memanggil fungsi Inisialisasi(). Pada baris 68–94, digunakan perulangan do-while untuk menjalankan menu secara terus-menerus sampai pengguna memilih keluar. Pada baris 69–73, ditampilkan menu pilihan operasi queue seperti insert, delete, cetak, dan keluar. Pada baris 75, pengguna memasukkan pilihan menu menggunakan cin. Pada baris 77–92, digunakan struktur switch-case untuk menjalankan perintah sesuai pilihan pengguna. Pada baris 78–80, jika memilih insert, program akan memanggil fungsi INSERT(). Pada baris 81–83, jika memilih delete, program akan memanggil fungsi DELETE(). Pada baris 84–86, jika memilih cetak, program akan memanggil fungsi CETAKLAYAR(). Pada baris 87–89, jika memilih keluar, program menampilkan pesan bahwa program selesai.Pada baris 90–91, jika pilihan tidak sesuai, maka program menampilkan pesan kesalahan. Pada baris 94, perulangan akan terus berjalan selama pilihan tidak sama dengan 4 (keluar). Pada baris 96–97, program diakhiri dengan return 0 yang menandakan program selesai dijalankan dengan baik.

LINK GITHUB

https://github.com/Arthur-369/PRAKTIKUM-Algoritma-dan-Struktur-Data